LocalCloudManager

System Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document describes implementation ownership, architecture, persistence, security and operational boundaries. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-LocalCloudManager-java-spring. 1 detailed Java SysDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-LocalCloudManager-go.

LocalCloudManager - Go 5.2.2

Standalone support system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/LocalCloudManager. Entry point: ./cmd/localcloudmanager. Module: arrowhead/ah5-support-go/localcloudmanager. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. Existing role selection and API behavior are retained. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed. See [Graphical orchestration and security editor](GRAPHICAL-ORCHESTRATION.md) for local rule and policy editing.

Listening port

Default 8480 on all interfaces. Override with SERVER_PORT=N (or PORT=N) or --server.port N. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Detailed implementation evidence

1 native Go SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-LocalCloudManager-rust.

Rust LocalCloudManager

Current implementation - secure Rust candidate cloud (2026-10-17)

The LocalCloudManager executable now uses src/secure_main.rs and src/secure.rs, not the historical HTTP launcher/router described below. It requires --config CONFIG.json. Automatic legacy binary discovery is disabled. The manager depends on the parent Rust support library for verified TLS, expected-provider CN/profile pinning, certificate-bound identity login/token refresh and bounded HTTPS requests. Deployed console: https://127.0.0.1:10590/. System API: https://127.0.0.1:10520/localcloud/ (mandatory mTLS). Deployment: Arrowhead-test-cloud-rust/candidate-native/, project arrowhead-rust-candidate. The older Rust cloud remains separate. During final verification it was recreated by a separate deployment change and its UI remapped from8590to10490; this work did not modify that project. The browser console uses HTTPS plus a private-password login and Secure/HttpOnly/SameSite cookie. It does not require a browser client certificate. The candidate CA is private, so browsers may show a certificate warning; no global trust store was changed. Password is in candidate-native/secrets/localcloudmanager/console-password (private file; never logged). The manager's system listener requires a verified local sy/op certificate and a matching verified sysop identity token for management. Echo is certificate-only. See docs/SECURE-DEPLOYMENT.md for the current configuration, service contracts, permission boundaries, test evidence and limitations. The manager now makes real revision-checked SysMonitor subscription writes, forwards plant POST/PUT/PATCH/DELETE correctly, manages simple-store rules, reads diagnostics/historian data and uses a scoped Rust Docker gateway rather than receiving a Docker socket. The Markdown below, the relocation source manifest, the earlier generated LaTeX/PDF catalogue and docs/legacy/ record the superseded relocation snapshot, not the active deployed secure contract. They do not supersede the new secure deployment documentation. Full Java equivalence, telemetry broker publishing and the cloud's missing federation/onboarding capabilities are still not claimed.

Historical relocation record (superseded runtime)

Canonical relocated source: ah5-support-rust/LocalCloudManager. Target system version5.2.2; associated services localCloudManagement and legacy monitor, both1.0.0. The latest located cloud/workspace manager sources matched SHA256 c3150daa4dd91e854439e31626895c5ff86e77881c338c5281306bda49f44d6b before relocation. This is a source/documentation relocation, not a Java-parity upgrade or secure deployment. The existing Arrowhead-test-cloud-rust/core-systems copy remains a deployment compatibility snapshot because other roles share its runtime and build context. No live source, cloud, container, key or state was removed or restarted. Develop against this canonical package instead of editing both copies independently.

Critical boundary

The inherited manager is a development HTTP adapter: no native inbound TLS, no configured mTLS provider identity and no native AMQP sysMonitor publisher. It sends/accepts legacy Bearer SYSTEM//... identities in its shared runtime. Those are not cryptographic authentication. Do not expose this package directly or assume it meets the newer Rust support runtime's certificate-authenticated contract. A TLS proxy alone does not correct legacy provider authentication. See COMPATIBILITY.md before use. The dedicated launcher defaults to http://127.0.0.1:8480/ instead of the inherited all-interface bind. This is a packaging safeguard, not a complete security fix. Historical UI8590/API8580 mappings belong to cloud deployment forwarding and are not created here.

Build and source layout

This is an independent nested Cargo package, with its own Cargo.toml/lock and workspace boundary. Root Makefile build/test/check targets explicitly include it. Root make localcloudmanager builds it; make localcloudmanager-docs generates/compiles its documentation. The generic parent arrowhead-support launcher does not select this legacy manager profile. src/main.rs is the dedicated launcher, rejecting roles other than localcloudmanager without mutating process environment. src/localcloudmanager.rs retains the existing Axum console handlers and compile-time embedded HTML/CSS/JavaScript in src/ui. api.rs, registry.rs and state.rs are

the inherited shared runtime dependencies; retaining them avoids a dependency on the old checkout. Other role implementations remain source dependencies and are not selected by this executable. No Java/Go/Node process implements the manager. Original manager business logic and UI are retained; deliberate packaging changes are the crate name/workspace boundary, library exports, role-default configuration helper, dedicated launcher and loopback default. Additional tests, documentation and source-provenance records are provided. The legacy src/ui.html remains an unused historical asset; active UI uses src/ui/index.html/app.js/styles.css.

Configuration

Environment	Actual source behavior
ARROWHEAD_ROLE	Omit or use localcloudmanager; other values rejected by dedicated launcher
SYSTEM_NAME, DOMAIN_NAME	Default LocalCloudManager/localcloudmanager; used as declared identity/advertised address, not validated certificate identities
SERVER_PORT, LISTEN_ADDRESS	Default8480 and loopback; explicit listen address can override it, but exposes inherited insecure transport
DATA_FILE	Independent JSON state, default /data/state.json; use a private dedicated path, never existing active cloud state
SERVICE_REGISTRY_URL, AUTHENTICATION_URL	Inherited HTTP defaults; HTTPS URLs alone do not provision a client certificate, expected-CN pin or identity-token login
DATAMANAGER_URL, SIMPLE_ORCHESTRATION_URL, SYSMONITOR_URL, PLANT_DESCRIPTION_ENGINE_URL	Provider adapter bases; shapes must be qualified against actual deployed providers
MANAGEMENT_WHITELIST, TOKEN_SECRET	Shared legacy runtime settings, not the console session boundary or proof of secure identity
CONSOLE_PASSWORD_HASH	scrypt:base64salt:base64hash, N16384/r8/p1; salt>=16bytes/hash>=32bytes; missing or invalid value returns503for console APIs
CONSOLE_ALLOWED_HOSTS	Host-name allowlist, default127.0.0.1,localhost,[::1]; ports are ignored
CONSOLE_SESSION_SECONDS	Default28800; positive bounded values should be provisioned; inherited implementation lacks robust range validation
DOCKER_SOCKET, COMPOSE_PROJECT_NAME	Unix Docker socket and project filter; static source allowlist controls lifecycle; MANAGED_SERVICES is not used

Missing password fails closed for /api/ but does not disable static assets, health or legacy Arrowhead routes. Sessions and history reset on restart. Keep this process in an isolated development environment with explicit trust boundaries.

Documentation and provenance

../documentation/LocalCloudManager/v5.2.2/ contains SysD, SysDD, two SDs and two HTTP IDDs, plus a combined catalogue. No sysMonitor AMQP SD/IDD is fabricated for an unimplemented publisher. The revised SysML variability model in docs/LocalCloudManager.sysml records Java-cloud, Go and Rust variants and the actual Rust qualification boundary. Historical Markdown/SysML is preserved unchanged under docs/legacy, not endorsed as evidence of interchangeability. Run from this module. The original source defaults to ../../Arrowhead-test-cloud-rust/core-systems for read-only source hashing; override REFERENCE when needed. docs/source-manifest.json records original and relocated hashes plus the route inventory. Read

VERIFICATION.md for actual test/build evidence. The repository's original Arrowhead LaTeX class/logo is reused.

Listening port

System API default 127.0.0.1:8480. Override with "listen" in CONFIG.json, SERVER_PORT=N or --server.port N. The HTTPS console listener ("consoleListen") is configured separately. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Detailed implementation evidence

1 native Rust SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-LocalCloudManager-java-spring/documentation/v5.2.2/LocalCloudManager-v5.2.2-SysDD.tex

- Go: Eclipse-Arrowhead-go/ah5-LocalCloudManager-go/documentation/LocalCloudManager/v5.2.2/LocalCloudManager-Go-v5.2.2-SysDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/LocalCloudManager/v5.2.2/LocalCloudManager-Rust-v5.2.2-SysDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

LocalCloudManager

System Design Description

Abstract
This document describes the design and implementation of the LocalCloudManager support system of Arrowhead 5.2.2 (`ah5-LocalCloudManager-java-spring`).

Contents

1 Overview **3**

2 Implementation **4**
2.1 Implementation language and tools 4
2.2 Functional properties implementation 4
2.3 Non functional properties implementation 5
2.4 Deployment 6
2.5 Local modifications 6
2.6 Verification 6

3 Services **7**

4 References **7**

5 Revision History **9**
5.1 Amendments 9
5.2 Quality Assurance 9

1 Overview

This document describes the LocalCloudManager system implementation, which realizes the LocalCloudManager SysD [1]. LocalCloudManager is the operator console of the local cloud: it shows the state of the cloud and carries out the management actions an operator starts, with its own Arrowhead identity. In Section 2, we describe implementation details of the system.

2 Implementation

This implementation is based on the SysD document LocalCloudManager [1] and is located in `ah5-LocalCloudManager-java-spring` of the primary Java source tree (`Eclipse-Arrowhead-java`). It is a Java-Spring support system built from two Maven projects: the console and Arrowhead system (artifact `arrowhead-localcloudmanager`, `localcloudmanager.jar`) and the Docker proxy (`docker-proxy/`, artifact `arrowhead-localcloudmanager-docker-proxy`).

2.1 Implementation language and tools

- Java 21, Spring Boot 3.4 (Web, Security, WebFlux client, Quartz), Log4j2, BouncyCastle (scrypt).
- Eclipse Arrowhead common library as a private vendored source copy (`vendor/`, the variant without JPA and log database): system information, ServiceRegistry registration, outsourced authentication and management filters, service collector, sysMonitor publisher, security profile guard.
- Storage: none (no database).
- Docker proxy: minimal Spring Boot Web application, Unix-socket HTTP client of the Docker Engine API.

2.2 Functional properties implementation

The following components implement the system (package `eu.arrowhead.localcloudmanager`):

Component	Responsibility
`LocalCloudManagerMain`, `LocalCloudManager SystemInfo`	Application; system `LocalCloudManager` (metadata `role`, `managedProject`, `consumerInterfaceTemplates`, X.509 public key) and the services `localCloudManagement` and `generalManagement`.
`init/LocalCloudManager ApplicationInitListener`	Registration by the common listener; registration state for `/health` and the overview.
`console/ ConsoleConnectorConfig`	Second Tomcat connector (plain HTTP, `console.port`, `console.address`).
`console/ConsoleFilter`	Separates the connectors: on 8490 the static UI (`ui/public`, packaged as `console/`), `/health`, proxy-secret, Host/Origin and password-configured checks, marks the Arrowhead filters as done; on 8491 only `/api`, `generalManagement` and the API documentation.
`console/ ConsoleSecurityConfig`	Spring Security: console chain (session cookie as authentication `ROLE_OPERATOR`, `/api` except `/api/ session` needs it, headers, stateless) and Arrowhead chain (HTTPS only). Replaces the common `DefaultSecurity Config`.
`console/ConsoleAuth`	Console login: scrypt password hash, sessions, per-client lock-out, Host/Origin check, proxy secret.
`api/http/ ConsoleApiController`	All `/api` routes and `/health`, matched in the order of the native console.
`api/http/ GeneralManagementAPI`	`generalManagement get-config`.
`security/Manager AccessFilter`, `Manager AuthorizationService`	On 8491, after authentication: Sysop, `lcm.api.whitelist` or ConsumerAuthorization grant of the `localCloud Management` operation (cached 30 s, fail closed).

Component	Responsibility
`client/OrchestratedProviderGateway`	Providers selected by the common service collector (orchestration on behalf of LocalCloudManager); HTTPS with the identity token; status and body passed through for LocalCloudConfigurator.
`client/TlsContextFactory`	Client TLS context: system certificate, cloud truststore, CRL of the cloud CA (rebuilt when the CRL file changes).
`client/DockerGateway`	HTTP client of the Docker proxy.
`service/ManagerService`	Views and actions: overview, registry management, DataManager, consumers, inter-cloud, sysMonitor, plant descriptions and SoS, orchestration rules and history, policies, logs, lifecycle, floorplans, local-cloud configuration.
`service/Inventory`	Inventory names, orchestration catalog, rule validation, registry management, messaging overview, SenML resolution, Docker log frames.
`service/PolicyEditor`, `RuleEditor`, `SosModel`	Policy grant/replace/revoke with snapshot checks; rule priority (native or create-and-remove); SoS validation, inclusion and cycle checks.
`service/CrlStatus`, `DockerHealthIndicator`	Remaining CRL validity; sysMonitor health (degraded while Docker is unreachable).
`docker-proxy/Allowlist`, `ProxyController`, `DockerSocket`	Docker proxy: decision per request, forwarding of allowed requests to `/var/run/docker.sock`.

A console request passes the `ConsoleFilter` (proxy secret for `/api`, Host/Origin, password configured), the Spring Security console chain (session) and, for changes, the confirmation check before the route handler calls the providing systems. A request on the Arrowhead connector passes the common authentication filter (client certificate and identity token), the management filter (`generalManagement`) and the `ManagerAccessFilter`.

2.2.1 Data handled and stored

None durable. Console sessions, the identity token, orchestration results (300 s) and the last 60 overview samples are kept in memory. SoS compositions, policies, rules and configurator designs are stored by their providing systems.

The interfaces exchange JSON objects built at run time; their structure is specified in the IDD documents.

2.3 Non functional properties implementation

2.3.1 Security

The console process has no Docker socket; Docker is reached only through the separate proxy on the internal `control` network, which forwards only the listed requests (SysD, Section Security). The console connector is published on the host's loopback address only. The security profile guard refuses insecure settings at start-up. Both containers run with a read-only root filesystem and without capabilities; the console as UID 10001.

2.3.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

2.3.3 Internal monitoring

Logs go to the console and a rolling file (Log4j2, no database appender). The common publisher sends the `sysMonitor` observations every 30 s (see the sysMonitor IDD); the echo is degraded while the Docker proxy cannot be reached.

2.3.4 Configuration

Spring properties (listed in the SysD), with secrets from `configtree:/run/secrets/`.

2.4 Deployment

The local `compose.yaml` (project `ah5-localcloudmanager`) builds and runs two containers from this folder:

- `cloud-monitor-api` (image `arrowhead-localcloudmanager-standalone:5.2.2`): ports 8490 (published on `127.0.0.1`) and 8491, on the external cloud network and the internal `control` network; key store, truststore and CRL mounted read-only; secrets store password, identity password and console password hash.
- `docker-proxy` (image `arrowhead-localcloudmanager-docker-proxy-standalone:5.2.2`): port 2375 on the `control` network only, Docker socket mounted read-only, `COMPOSE_PROJECT_NAME` and `MANAGED_SERVICES`.

The earlier combined test cloud (`Arrowhead-test-cloud-java`) is a separate deployment consumer and still runs the earlier packaging until it is redeployed.

2.5 Local modifications

Notes on this implementation in the workspace:

- Workspace system; operation names are camelCase (see the SD).
- `session`, `login` and `logout` are console-only and no longer registered; console endpoints for policies, rule priority, SoS, local-cloud configuration and chiplet floorplans are not part of the registered operation list.
- `generalManagement` has no `get-log` (no log database).

2.6 Verification

`mvn clean verify` runs the JUnit tests of both projects: a full-application MockMvc test of every `/api` route on both connectors (console login flow, Host/Origin refusal, confirmation header, pass-through of LocalCloud-Configurator, identity and grant checks on 8491, `generalManagement`); a real-Tomcat test that the console connector listens on the loopback address only and serves `/api` only after login; console authentication, CRL status, inventory, policy, rule-priority and SoS tests (ports of the native Node tests); the security profile test; and the Docker proxy allowlist (MockMvc with a stub socket). The browser models in `ui/test` are Node tests of the UI code.

3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`localCloudManagement`	[1]	[2]	[3]
`sysMonitor`	[1]	[4]	[5]
`generalManagement`	[1]	–	–
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
serviceDiscovery; serviceRegistry Management	[6]	[7]	–
identity	[8]	[9]	–
serviceOrchestration; service OrchestrationHistoryManagement	[10]	[11]	–
serviceOrchestrationSimpleStore Management	[12]	[13]	–
authorizationManagement (query, grant, revoke policies)	[14]	[15]	–
dataHistorian	[16]	[17]	–
sysMonitorHistory; sysMonitor Management	[18]	[19]	–
plantDescriptionManagement	[20]	[21]	–
gatekeeperManagement; gate TunnelManagement	Gatekeeper, GateTunnel	–	–
Docker Engine subset	Docker proxy	–	–
localCloudConfiguration	[22]	[23]	[24]
floorplanDiscovery; floorplan Management	Chiplet Floorplan Generator	–	–

4 References

[1] J. Delsing, “LocalCloudManager – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-LocalCloudManager-java-spring/documentation/v5.2.2/LocalCloudManager-v5.2.2-SysD.pdf.

[2] ——, “localCloudManagement – Service Description (LocalCloudManager),” Eclipse Arrowhead, SD 5.2.2, 2026, localCloudManager-v5.2.2-localCloudManagement-SD.pdf.

[3] ——, “localCloudManagement HTTP/TLS/JSON – Interface Design Description (LocalCloudManager),” Eclipse Arrowhead, IDD 5.2.2, 2026, localCloudManager-v5.2.2-localCloudManagement-HTTP-IDD.pdf.

[4] ——, “sysMonitor – Service Description (LocalCloudManager),” Eclipse Arrowhead, SD 5.2.2, 2026, localCloudManager-v5.2.2-sysMonitor-SD.pdf.

[5] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (LocalCloudManager),” Eclipse Arrowhead, IDD 5.2.2, 2026, localCloudManager-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[6] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[7] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[8] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[9] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/authentication/documentation/v5.2.2/Authentication-v5.2.2-identity-SD.pdf.

[10] ——, “DynamicServiceOrchestration – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-SysD.pdf.

[11] ——, “serviceOrchestration – Service Description (DynamicServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-dynamic/documentation/v5.2.2/DynamicServiceOrchestration-v5.2.2-serviceOrchestration-SD.pdf.

[12] ——, “SimpleStoreServiceOrchestration – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-SysD.pdf.

[13] ——, “serviceOrchestrationSimpleStoreManagement – Service Description (SimpleStoreServiceOrchestration),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/serviceorchestration-simple/documentation/v5.2.2/SimpleStoreServiceOrchestration-v5.2.2-serviceOrchestrationSimpleStoreManagement-SD.pdf.

[14] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

[15] ——, “authorizationManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-authorizationManagement-SD.pdf.

[16] ——, “DataManager – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-support-java-spring/DataManager/documentation/v5.2.2/DataManager-v5.2.2-SysD.pdf.

[17] ——, “dataHistorian – Service Description (DataManager),” Eclipse Arrowhead, SD 5.2.2, 2026, dataManager-v5.2.2-dataHistorian-SD.pdf.

[18] ——, “SysMonitor – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-support-java-spring/SysMonitor/documentation/v5.2.2/SysMonitor-v5.2.2-SysD.pdf.

[19] ——, “sysMonitorHistory – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorHistory-SD.pdf.

[20] ——, “PlantDescriptionEngine – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-support-java-spring/PlantDescriptionEngine/documentation/v5.2.2/PlantDescriptionEngine-v5.2.2-SysD.pdf.

[21] ——, “plantDescriptionManagement – Service Description (PlantDescriptionEngine),” Eclipse Arrowhead, SD 5.2.2, 2026, plantDescriptionEngine-v5.2.2-plantDescriptionManagement-SD.pdf.

[22] ——, “LocalCloudConfigurator – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, ah5-LocalCloudConfigurator-java/documentation/v5.2.2/LocalCloudConfigurator-v5.2.2-SysD.pdf.

[23] ——, “localCloudConfiguration – Service Description (LocalCloudConfigurator),” Eclipse Arrowhead, SD 5.2.2, 2026, localCloudConfigurator-v5.2.2-localCloudConfiguration-SD.pdf.

[24] ——, “localCloudConfiguration HTTP/TLS/JSON – Interface Design Description (LocalCloudConfigurator),” Eclipse Arrowhead, IDD 5.2.2, 2026, localCloudConfigurator-v5.2.2-localCloudConfiguration-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): location, new console modules, standalone deployment, consumed services and tests updated.	Claude Opus 5.5 (draft)
3	2026-11-09	5.2.2	Java-Spring implementation: components, connectors (8490 console, 8491 Arrowhead), Spring Security console login, Docker proxy project, orchestrated providers including LocalCloudConfigurator, deployment and JUnit/MockMvc verification. Node.js/nginx descriptions removed.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Architecture

The operator reaches the static dashboard and console API on the Go HTTP server. API handlers use the adapter's own identity for HTTPS provider calls. Runtime inspection/control uses a Unix-socket Docker client. Optional telemetry is published directly to AMQP. The package contains the original shared Go runtime because TLS/client/model/registration helpers are interdependent; the dedicated executable rejects other role selections.

2 Components

`cmd/localcloudmanager/main.go`: role-constrained launcher. `internal/cloud/localcloudmanager.go`: sessions, API aggregation, provider adapters, Docker operations. `assets.go` and `ui/`: compile-time embedded UI, retaining explicit UI_ROOT and legacy on-disk fallbacks. `app.go`: environment, server, registration and provider-call retry. `security.go`: certificates, CRL and identity. `monitoring.go`: AMQPS publisher. `model.go`: inherited JSON state. Other shared files remain compile-time dependencies, not newly deployed support systems.

3 Request sequence and trust boundaries

Transport mTLS is enforced when secure mode is selected. Static assets and health need no operator session. For `/api/`, Host/Origin policy and configured console password are checked, session requests are handled, then a valid operator cookie is required. Mutations additionally require `X-Arrowhead-Action:confirm`. These handlers run before the shared Arrowhead token/management dispatch; a bearer token does not replace a browser session. Provider calls obtain the manager's own token and refresh once after401. Network effects and local state are not a distributed transaction.

4 State and recovery

Console sessions and the last60 overview samples live in process memory and are lost on restart. Shared JSON state uses a temporary sibling file and rename, but no new fsync/process-lock guarantees are introduced. Registration retries every20seconds. Displayed registered/naming flags are synthetic and not evidence of successful registration. Provider/Docker failures appear in collected responses or502. Do not share active cloud state files with this package.

5 Security and configuration

Authentication server CN/profile are explicitly pinned before credentials; other provider peers have CA/SAN validation without universal expected-CN pinning. Optional CRLs are checked for new connections. Console cookies are HttpOnly/SameSite=Strict but not Secure or port-isolated; Origin checks do not compare exact ports. Lifecycle uses the static systemToCompose allowlist (MANAGED_SERVICES is not consumed); log reads are project-scoped, not restricted to that allowlist. A raw Docker socket is a privileged trust boundary; deployment isolation or a least-privilege proxy remains necessary. See README.md for every environment variable and COMPATIBILITY.md for release blockers.

6 Compatibility and release gates

This relocation intentionally preserves the latest located manager's business behavior. New changes are packaging (standalone nested module/role-specific launcher), embedded UI fallback, documentation and tests. This is not a security remediation or full Java-equivalence implementation.

1. `PUT/DELETE/api/sysmonitor/subscriptions/{providerSystem}` currently checks confirmation and returns updated/removed flags without forwarding a write. Do not interpret this as applied configuration. This remains a release blocker.

2. `/api/intercloud` returns a static unavailable/no-relay summary. It does not inspect Gatekeeper/GateTunnel. Consumer cloud labels include hard-coded TestCloud strings; no live foreign authorization catalogue is queried.

3. Overview hard-codes registration/naming flags and leaves several collections empty. Subscription status hard-codes intake.connected=true/providerFound=false. These are display adapters, not authoritative readiness or security evidence.

4. Historian, SysMonitor, plant and orchestration adapters target the inherited Go-cloud routes/shapes. They are not proven compatible with the newer generic Go support port or latest Java APIs. The standalone module must be qualified against the providers actually deployed.

5. Console Host/Origin checks compare host names, not exact origins/ports. The `ah_console` cookie is not port-isolated and lacks Secure. Sessions/history are in-memory and reset on restart. Missing password disables APIs. Login lockout is global, five failures in five minutes then one-minute lockout; this is not comprehensive abuse protection.

6. The manager API is dispatched before the shared application's token/management checks. Its boundary is console password/session plus the transport's mTLS if enabled, not a bearer-token-only API. Static assets and health have no console session requirement. A proxy must preserve this intended boundary.

7. Authentication has explicit CN/profile pinning. Other provider clients use CA/SAN validation but do not generally pin expected provider CN. Do not claim universal authority-substitution resistance for this inherited code. Redirects are disabled. Optional CRLs reload for new connections only; omitted CRLs mean no revocation enforcement.

8. Lifecycle uses the static systemToCompose service allowlist and Compose-project filtered container list; MANAGED_SERVICES is not consumed by this implementation. Action is start/stop/restart only. Logs, however, can read any returned project container, not just the lifecycle allowlist. The raw Docker socket still confers broad daemon privilege: application checks are not a least-privilege socket boundary. Production needs a constrained proxy and deployment review.

9. Shared runtime source includes other roles for compile-time dependencies and inherited tests. The dedicated executable refuses selecting those roles; they are not a second support-system deployment. No Docker socket or live state is needed by tests.

10. Telemetry AMQP publisher has no durable local outbox; retries/reconnects do not establish exactly-once delivery. power=line/available and process health are generated status, not measurements of physical hardware. Registration is retried every20seconds; displayed registration flags are not confirmation. No real broker test is claimed here.

11. Shared state is the inherited JSON format, not the newer support port's transactional state model. There is no new Java SQL migration, encryption-at-rest or multi-process ownership guarantee. Keep independent state and backups.

12. Historical Markdown/SysML claims of three-way interchangeability are retained only under docs/legacy. The revised model and new SysD/SysDD/SD/IDD set distinguish intended common contracts from actual Go qualification.

13. The shared serviceOperations helper generates POST/name metadata for localCloudManagement and the legacy monitor service, whereas actual handlers include GET/PUT/PATCH/DELETE and nested paths. Registration metadata is therefore not an authoritative HTTP contract. The new IDDs document actual handler methods/paths; correcting advertised metadata is separate functional work, not silently folded into relocation.

Source relocation does not modify running containers or remove their deployment source. Before switching deployment build paths, close these gaps, qualify provider DTOs/policies, review Docker access, isolate browser trust domains, and test rollback.

7 References and verification

Authoritative moved code: LocalCloudManager/internal/cloud and embedded ui sources. Exact original/relocated SHA256 hashes are in docs/source-manifest.json. docs/legacy preserves the earlier local Markdown/SysML, not an endorsement of its parity claims. The revised SysML variability model is descriptive and has not been checked by a SysML parser. Executed checks and qualification limits are recorded in VERIFICATION.md. Source movement does not deploy or alter running clouds.

1 System design

1.1 Components and boundaries

The standalone executable runs Tokio and Axum, with request-ID, trace and panic-catching layers. The shared Config/App/Store model remains local to this package. The manager UI is already embedded with include_str; it needs no old checkout or Node server at runtime. Shared API, registration and state modules are retained so the package can build independently. The dedicated launcher rejects other ARROWHEAD_ROLE values.

The browser uses `/api/` console handlers. Host/Origin names and configured password are checked before session access. Session management is the exception to the authenticated-cookie requirement. Mutations additionally check `X-Arrowhead-Action:confirm`. Static assets and `/health` are not console-authenticated. Legacy Arrowhead routes are separate and accept declared SYSTEM identities through the inherited requester/-manager path.

Provider adapters use a default reqwest Client and the manager's declared SYSTEM name. There is no configured client certificate, identity-token login, expected-CN pinning or explicit per-request timeout. Transport is HTTP unless an HTTPS provider URL is supplied, and HTTPS does not provision mTLS. The listener itself is plain HTTP. This is not equivalent to the parent Rust support runtime's secure transport.

1.2 Collection and mutations

Overview queries registry systems/services and Docker containers concurrently, groups runtime state against known system names, and keeps the last60overview samples in memory. Several output collections/flags are placeholders; registration and naming booleans are not authoritative.

DataManager is queried through the inherited data-historian system/service/data paths and flattened for the dashboard. Rules use simple-store query/create/remove; creation validates consumer, service instance and positive priority against an inferred interoperability catalogue before forwarding a candidates array. Provider DTOs must be qualified against the versions actually deployed.

Plant creation/update/removal forwards to the inherited plant-description-engine path. PATCH currently forwards PUT, and create currently returns200. SysMonitor read errors are hidden behind empty arrays; subscription mutation acknowledges without forwarding. Intercloud is a static unavailable summary, not an active federation monitor.

Docker control uses a hand-written HTTP-over-Unix-socket client, supporting chunked replies and multiplexed logs. Lists carry a Compose project-label filter. Lifecycle accepts only start/stop/restart and static allowlisted services. Logs can include any project container. The raw socket remains a daemon-privilege boundary; there is no new constrained proxy, response-size limit or timeout.

1.3 Registration, state and recovery

Registration retries every3seconds until one successful pass, then stops. It advertises localCloudManagement and legacy monitor1.0.0 over generic_http with policy NONE. Handler session/plant/subscription mutation routes are not all represented in registration metadata. No sysMonitor AMQP service or publisher is present.

Console sessions and history are process-global in-memory structures; restart loses them. Password verification uses scrypt under a mutex. Sessions expire on access; repeated failures cause a one-minute lockout. Cookie/Origin rules do not isolate localhost ports. Configure sensible session lifetimes because range validation remains incomplete.

The shared JSON Store reads corrupt JSON as default empty state and writes through a temporary sibling/rename. It does not provide the newer parent support runtime's process-lock/fsync guarantees. Network mutations are not atomic with local state. Use an independent private state file; this package must never be pointed at active cloud state as a migration mechanism.

1.4 Variants and deployment

The revised SysML model distinguishes the Java-cloud Node.js adapter, native Go package and this Rust development package. It describes intended variability, not proven interchangeability. Original source remains in the cloud build context as an unchanged compatibility snapshot. This relocation does not rebuild/restart containers or create the historical UI8590/API8580 mappings.

Before deployment, implement secure identities/transport, close no-op and placeholder behavior, qualify provider contracts and runtime privileges, and test rollback. A TLS proxy alone does not fix legacy provider authentication. See COMPATIBILITY.md for the full inherited limitation list and VERIFICATION.md for executed checks.

2 Compatibility and inherited limits

This relocated module is not the parent Rust support runtime's native mTLS implementation. Packaging does not establish full Java equivalence or new security hardening.

1. The listener is plain HTTP. The dedicated launcher now defaults to loopback, but explicit LISTEN_ADDRESS may expose it. No native inbound TLS/mTLS certificate verification exists in this source.

2. Provider calls use reqwest defaults and legacy `BearerSYSTEM//LocalCloudManager`, not authenticated identity-token login or a configured client certificate. No expected provider-CN pinning, CRL policy or explicit request timeout/response-size limit is added. HTTPS URLs alone do not solve this. Legacy management routes trust declared SYSTEM names.

3. `/api/sysmonitor/subscriptions/{provider}` PUT and DELETE only acknowledge updated=true after confirmation; neither forwards changes. Both methods return the same acknowledgement. This is not applied subscription configuration.

4. Intercloud is a static unavailable/no-relay view. Consumers have empty authorized/orchestrated collections and TestCloud labels. Overview includes synthetic registration/naming state and placeholder collections; subscription intake.connected=true is not broker readiness.

5. SysMonitor read failures are converted into empty data. Plant reads partly report provider failure, but all provider adapter paths/shapes still need current Java/Go/Rust interoperability testing. Plant PATCH is routed through a PUT forwarding handler. Plant POST currently returns200rather than201.

6. Console cookie `ah_console` is HttpOnly/SameSite=Strict but lacks Secure and localhost-port isolation. Host/Origin checks compare names without exact origin-port matching. Sessions/history are process-global in-memory state. Scrypt executes under a mutex on an async handler; abuse/capacity hardening is outstanding.

7. Invalid/missing password disables console APIs with503, but static UI/health and legacy interfaces remain accessible under their own weaker rules. A console session is not required for legacy declared-identity endpoints. Confirmation header is not cryptographic replay protection.

8. Docker uses a raw Unix socket with project-filtered list and static lifecycle allowlist. Logs may read any project container, not just lifecycle-allowed services. MANAGED_SERVICES is not consumed. Application checks do not constrain the daemon privilege conferred by the raw socket. Docker read/response parsing has no explicit timeout or size bound.

9. No native sysMonitor AMQP publisher or service registration exists here. Historical monitor1.0.0 is echo only, not the canonical telemetry contract. No claim of AMQPS publication, broker confirms or real broker qualification is made.

10. Registry metadata covers a subset of handler routes and advertises generic_http/policy NONE. Registration retries every3seconds until first success, then stops; no ongoing refresh is implemented. Displayed registered=true is synthetic.

11. Shared JSON state lacks a new process lock/fsync/encryption policy; corrupt JSON currently defaults to empty state. Do not open real deployment state with this package or use shared files between processes. No Java state migration is implemented.

12. The shared runtime retains additional roles for source dependencies and inherited tests. Only LocalCloudManager is selected by the dedicated executable. Original deployment source is retained unchanged so other roles/build contexts are not broken.

13. The revised SysML model describes intended variability, not proven substitution. Historical legacy docs claiming interchangeability are kept as provenance only. No SysML parser, full browser deployment suite, Java golden fixtures, vulnerability scan or production qualification is included.

Moving source and creating documentation does not resolve these limits. Close them and qualify security, provider DTOs, rollback and runtime isolation before selecting this package for a working cloud.

3 References and qualification

See `docs/source-manifest.json` for original/relocated source SHA256 records, `VERIFICATION.md` for checks actually run, and `docs/LocalCloudManager.sysml` for the descriptive, unparsed variability model. No Java interchangeability, native TLS or sysMonitor publisher is claimed. Existing cloud source and running deployment remain unchanged.